LECTURE 8: SCALABLE INDEXING for BIG DATA PROCESSING EXERCISES

DS 8001

Ex1: Hashing 1

Consider the hash function

$$h(x) = \sum_{i=0}^{n-1} 2^i x_{n-i} = 2^{n-1}x_1 + 2^{n-2}x_2 + \cdots + 2x_{n-1} + x_n,$$

which maps a binary vector $x = (x_1, \ldots, x_n) \in \{0,1\}^n$ to a non-negative integer.

Your goal is to generate K random binary vectors of size n, one at a time, and to store each distinct vector once in a set B: a vector that has already been generated is not added again.

Ex1: Hashing 1 - continued

The algorithm HashingLab below generates the vectors and builds B:

```
1: function [B] = HashingLab(n, K)
2:     Initialize B = {}
3:     for k = 1, ..., K
4:         Generate x̂ ∈ {0, 1}^n
5:         if x̂ ∉ B
6:             B.add(x̂)
```

(a) In which step (line) of HashingLab can the hash function h be used, and how? What is the potential benefit of using it there, compared with storing B as an unsorted list of vectors?

(b) Consider the alternative hash function $g(x) = \sum_{i=1}^{n} x_i$. Would you prefer to use g instead of h in HashingLab? Explain why.

Ex2: Hashing 2

The hash function h maps binary vectors of dimension m to binary vectors of dimension n, where $m > n$, with the help of an $n \times m$ matrix H whose entries are random bits (each entry is 0 or 1). More specifically, given the matrix H, the hash function $h : \{0,1\}^m \to \{0,1\}^n$ gives the following hash value for a key $x \in \{0,1\}^m$:

$$h(x) = Hx \bmod 2,$$

where Hx is the ordinary matrix–vector product and the mod operator is applied element-wise (to each entry of the vector Hx).

For instance, if $m = 3$, $n = 2$, H is the 2×3 matrix of all ones and x is the vector of all ones, then Hx is the vector of all threes, $(3,3)$. Applying mod 2 element-wise gives the two-dimensional vector $h(x) = (1,1)$, the vector of all ones.

Toronto Metropolitan University

EX2: HASHING 2 - CONTINUED

Work through the hash function h on your own example, with $n = 3$.

(a) **Key x.** Write your first name in lowercase English letters (drop accents, hyphens and spaces). If it has fewer than 5 letters, repeat it until it has at least 5, e.g., "vader", "obiwan", "lukeluke", "leialeia", "wuwuwu". Let m be the length of the resulting string and c_i its ith letter. Set $x_i = 1$ if c_i is one of the letters `a`, `b`, . . ., `m`, and $x_i = 0$ otherwise. (For instance, "mucahit" gives $m = 7$ and $x = (1, 0, 1, 1, 1, 1, 0)$.)

(b) **Matrix H.** Generate the random binary $3 \times m$ matrix H with NumPy, using the last 4 digits of your student number as the seed:

```
import numpy as np
seed = 1234   # last 4 digits of your student number
rng = np.random.default_rng(seed)
H = rng.integers(0, 2, size=(3, m))   # m from part (a)
```

(c) **Hash value.** Compute Hx, and then $h(x) = Hx \bmod 2$.

Ex3: Hashing 3

Recall the universal family from the lecture. Let n be a prime, split a key x into blocks $(x_1, \ldots, x_k)$ with $0 \le x_i < n$, draw $a_1, \ldots, a_k$ independently and uniformly at random from $\{0, 1, \ldots, n-1\}$, and set

$$h_a(x) = \left(\sum_{i=1}^{k} a_i x_i \right) \bmod n.$$

Here $k = 3$ and $n = 11$. Consider the two keys $x' = (3, 8, 5)$ and $x'' = (9, 2, 7)$.

(a) Compute $h_a(x')$ and $h_a(x'')$ for $a = (4, 7, 9)$. Do the two keys collide?

(b) Suppose $a_1 = 4$ and $a_2 = 7$ have been drawn, but a_3 has not. Find every $a_3 \in \{0, \ldots, 10\}$ for which $h_a(x') = h_a(x'')$, without trying all 11 values. (Hint: $2 \cdot 6 \equiv 1 \pmod{11}$.)

(c) Using (b), explain why $\Pr_a \{h_a(x') = h_a(x'')\} = 1/11$ when all three coefficients are drawn at random.

Ex3: Hashing 3 - continued

(d) Now let the modulus be the composite number $n = 12$: blocks satisfy $0 \le x_i \le 11$, and each a_i is uniform on $\{0, \ldots, 11\}$. Compute $\Pr_a \left\{ h_a(y') = h_a(y'') \right\}$ for

(i) $y' = (5, 1, 2)$ and $y'' = (5, 1, 6)$;

(ii) $y' = (5, 1, 2)$ and $y'' = (5, 1, 7)$.

Explain the difference between (i) and (ii), and why the lecture's theorem requires the modulus to be prime.

Ex4: Permutation-Based Indexing

An image collection $D = \{o_1, \ldots, o_8\}$ is indexed with $n = 4$ pivots $R = \{r_1, \ldots, r_4\}$. The table gives the distance from every object, and from a query q, to every pivot. The last column is the true distance $d(q, o_i)$; it is used only in stage 3 (exact re-ranking) and to measure recall.

	$d(\cdot, r_1)$	$d(\cdot, r_2)$	$d(\cdot, r_3)$	$d(\cdot, r_4)$	$d(q, \cdot)$
o_1	7.2	5.0	7.5	2.5	3.8
o_2	4.5	7.6	6.7	5.3	4.7
o_3	6.3	9.1	8.6	6.5	6.3
o_4	4.2	5.3	5.4	2.9	2.5
o_5	2.8	3.9	0.9	4.3	2.8
o_6	6.2	2.9	3.4	5.2	4.4
o_7	4.7	1.7	3.3	2.2	1.4
o_8	1.8	5.0	3.1	3.8	2.3
q	4.0	2.9	3.7	1.6	–

Ex4: Permutation-Based Indexing - continued

Throughout, use pruned lists of length $\tilde{n} = 2$, return $K = 2$ results, re-rank the $c = 3$ best candidates, and break ties in favour of the smaller index. $\text{pos}_q(r)$ and $\text{pos}_o(r)$ denote the positions of pivot r in $\tilde{L}(q, R)$ and $\tilde{L}(o, R)$. A posting (o, p) in the list of one of q's pivots r earns o $\tilde{n} + 1 - \max\{\text{pos}_q(r),\, p\}$ points. Recall@K is the fraction of the true K nearest neighbours that is returned.

(a) Write the pruned ordered list $\tilde{L}(o_i, R)$ of every object, and $\tilde{L}(q, R)$.

(b) Build the metric inverted file: the list of postings (o_i, rank) of each pivot. How many postings does it hold? Which lists does q read, how many postings is that, and which objects are the candidates?

(c) For every candidate, compute (i) its points, and (ii) the "tempting" score $\sum |\text{pos}_q(r) - \text{pos}_o(r)|$ taken only over the pivots r that both lists contain (smaller is better). For o_4 and o_7, also compute the score in which a pivot missing from a list is placed at position $\tilde{n} + 1 = 3$, and check that it equals $\tilde{n}(\tilde{n} + 1) - 2 \cdot$ points. Which three candidates does each of (i) and (ii) pass to stage 3?

EX4: PERMUTATION-BASED INDEXING - CONTINUED

(d) Re-rank each of the two selections from (c) by true distance and return the $K = 2$ closest. Find the true 2 nearest neighbours of q, and compute recall@2 in both cases. Which true neighbour does the points-based search miss, and at which stage is it lost? Would a larger c bring it back? Would it be a candidate with $\tilde{n} = 3$?

(e) **Prefix index** with prefix length $\ell = 2$: group the objects by their pruned lists. Which group does q read, and what is returned after re-ranking that group? What is recall@2? Why does the prefix index miss a neighbour that the metric inverted file found?

(f) **At scale**: $N = 10^8$ images, $n = 1000$ pivots, $\tilde{n} = 5$, $c = 100$, and inverted lists of similar length. How many distance computations does building the index take, and how many postings does it store? Per query, how many distance computations and roughly how many postings does the search need, compared with an exhaustive scan? How do these numbers change if $\tilde{n}$ is doubled?

Ex5: Distributed Search

A metric inverted file over $D = \{o_1, \ldots, o_8\}$, with $n = 6$ pivots and $\tilde{n} = 2$, is spread over $P = 2$ machines M_1 and M_2 and queried through a broker. The query has $\tilde{L}(q, R) = (r_2, r_5)$, so only two inverted lists matter:

$$r_2 \to (o_1, 1)\ (o_4, 1)\ (o_6, 2)\ (o_8, 2), \qquad r_5 \to (o_3, 1)\ (o_5, 2)\ (o_6, 1)\ (o_7, 2).$$

The true distances to q are:

	o_1	o_2	o_3	o_4	o_5	o_6	o_7	o_8
$d(q, \cdot)$	1.9	5.0	3.1	2.6	3.4	1.2	4.0	2.9

Points are as in Ex4: a posting (o, p) in the list of q's pivot r earns $\tilde{n} + 1 - \max\{\mathrm{pos}_q(r), p\}$. Return $K = 2$ results and break ties in favour of the smaller index.

(a) **Partition the objects**: M_1 stores $o_1, \ldots, o_4$ and M_2 stores $o_5, \ldots, o_8$, each with a local metric inverted file built with the same pivots. Each machine runs stages 1–3 on its own objects, re-ranking its $c = 2$ best candidates, and returns its $K = 2$ best objects with their true distances. What does each machine return, and what is the broker's final answer? What is recall@2?

Ex5: Distributed Search - continued

(b) Explain why it is enough for each machine in (a) to return only K results: show that an object in the overall top K of the re-ranked objects is always in its own machine's top K. What can this merge *not* recover?

(c) **Partition the index by pivot**: M_1 owns the complete lists of r_1, r_2, r_3 and M_2 those of r_4, r_5, r_6. The broker contacts the owners of q's pivots. Each owner sends the points of every posting in its list; the broker adds them up per object, re-ranks the $c = 3$ best objects and returns $K = 2$. Give the final answer. Then repeat the query assuming that each owner sends only its 2 best partial scores. What goes wrong, and why?

(d) **At scale**: $P = 20$ machines, $n = 200$ pivots (10 per machine under pivot partitioning), $N = 10^8$, $\tilde{n} = 4$, $K = 10$, and lists of similar length. For each strategy: how many machines does a query contact, how many items reach the broker per query, and how many postings cross the network while the index is built? When would you prefer each strategy?